

IEEE x Knight Hacks Embedded Workshop!

Session 3: Timers/Interrupts (Advanced)



[IEEE Discord Link](#)



[Github Link](#)

What's your level?

PREREQUISITE: You ***MUST*** have a laptop that can run the Arduino IDE.

Level 1: I don't know how to code and I don't know Embedded. :(

Level 2: I've never programmed a microcontroller before, but I know how to code!

Level 3: I know how to flash code to a board! I can write and upload firmware, use serial communication, understand GPIOs, and read datasheets without crying. I also know how to breadboard!

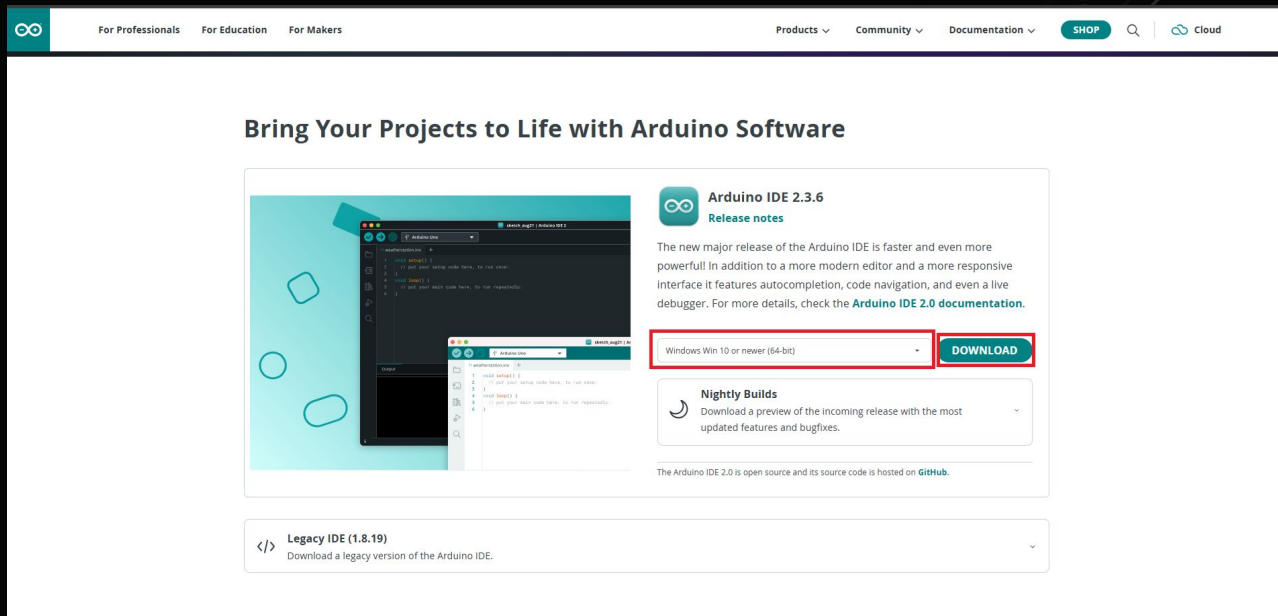
Summary of Topics Covered!

- How to install the Arduino IDE and configure it for use with ESP32 MCU
- How to connect the components on the breadboard
- Basics of embedded programming structure
- Basics of GPIO pins and usage
- Basics of ADC input and potentiometers
- How to put it together for LED control

How to Install and Configure the Arduino IDE

1. Install Arduino IDE
2. Install the “esp32 by espressif” module
3. Install the graphics modules (Optional for this one)
4. **IMPORTANT:** install the VCP drivers (Silicon Labs CP201X)

How to Install and Configure the Arduino IDE continued..



Bring Your Projects to Life with Arduino Software

Arduino IDE 2.3.6
Release notes

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger. For more details, check the [Arduino IDE 2.0 documentation](#).

Windows Win 10 or newer (64-bit) **DOWNLOAD**

Nightly Builds
Download a preview of the incoming release with the most updated features and bugfixes.

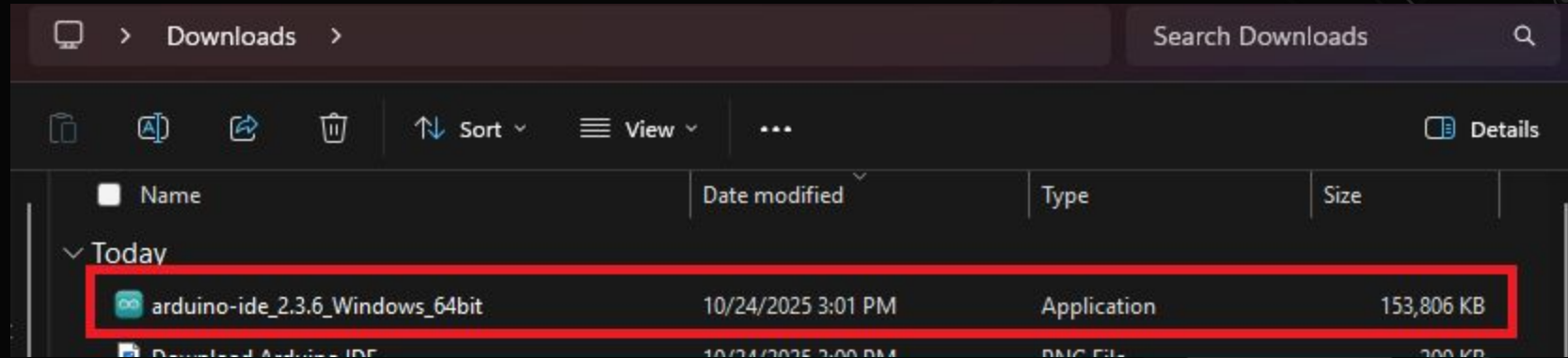
The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

Legacy IDE (1.8.19)
Download a legacy version of the Arduino IDE.

Select your OS and click “DOWNLOAD”

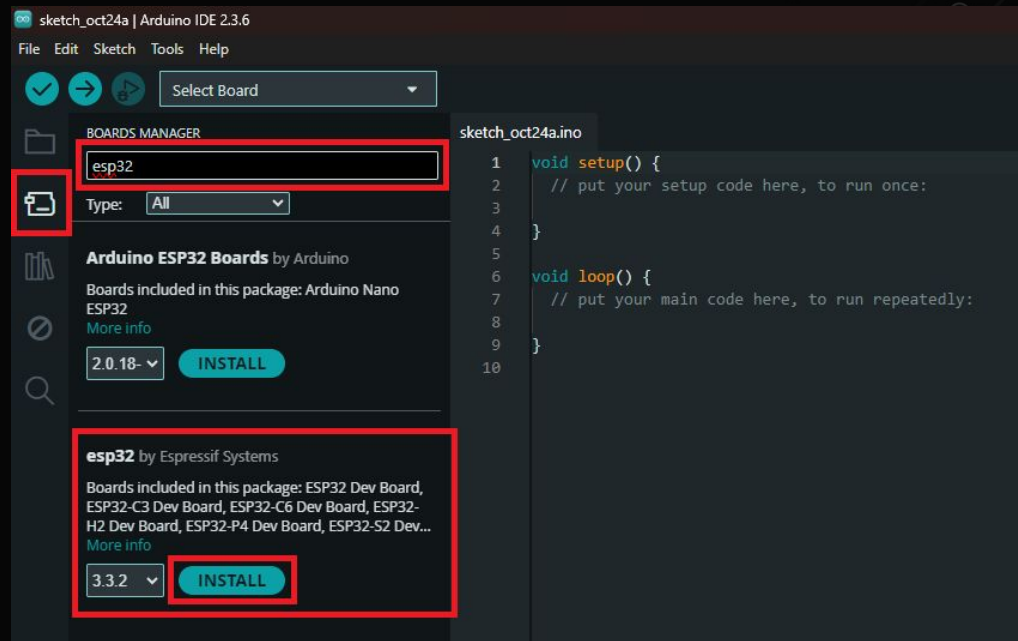
Link: <https://www.arduino.cc/en/software/>

How to Install and Configure the Arduino IDE continued...



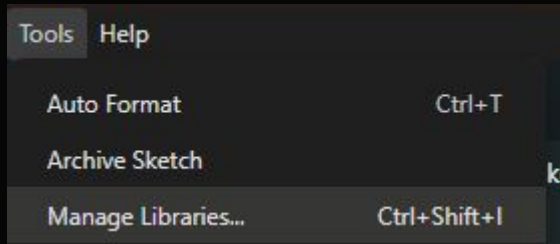
Next, run the installer!

How to Install and Configure the Arduino IDE continued...

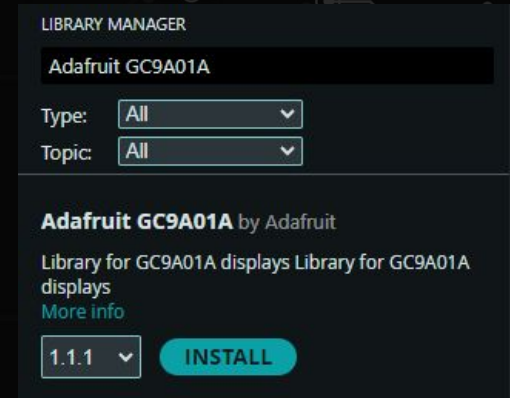


Open the IDE > Select the “Boards” tab > search “ESP32” > Install the “esp32” pack by Espressif Systems

How to Install and Configure the Arduino IDE continued...

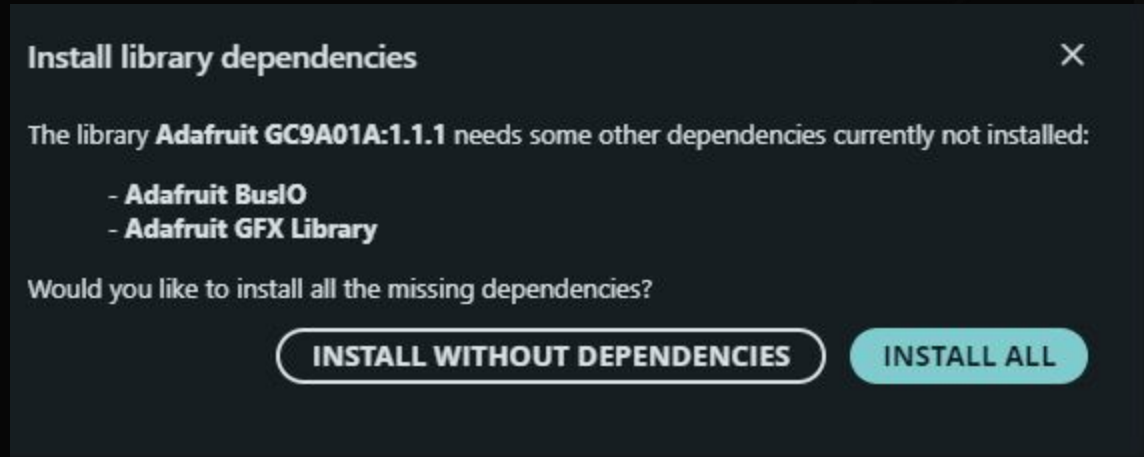


Once you load the IDE,
go to tools and go to
manage libraries.



You will see a library
manager pop up, in
the search bar, type in
"Adafruit GC9A01A"

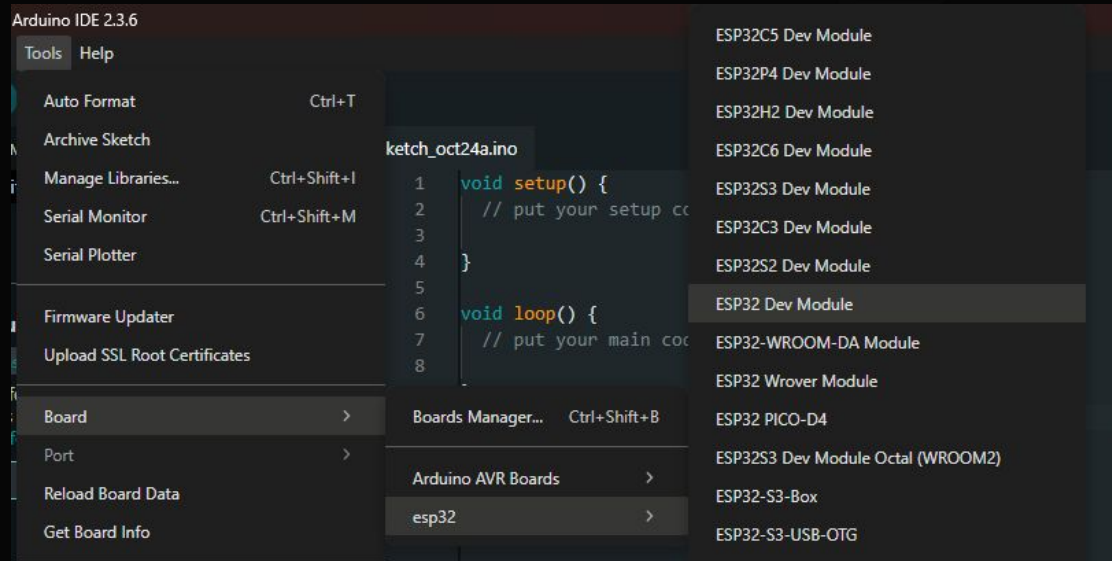
How to Install and Configure the Arduino IDE continued...



It will ask you to install 2 additional libraries, press **INSTALL ALL**.

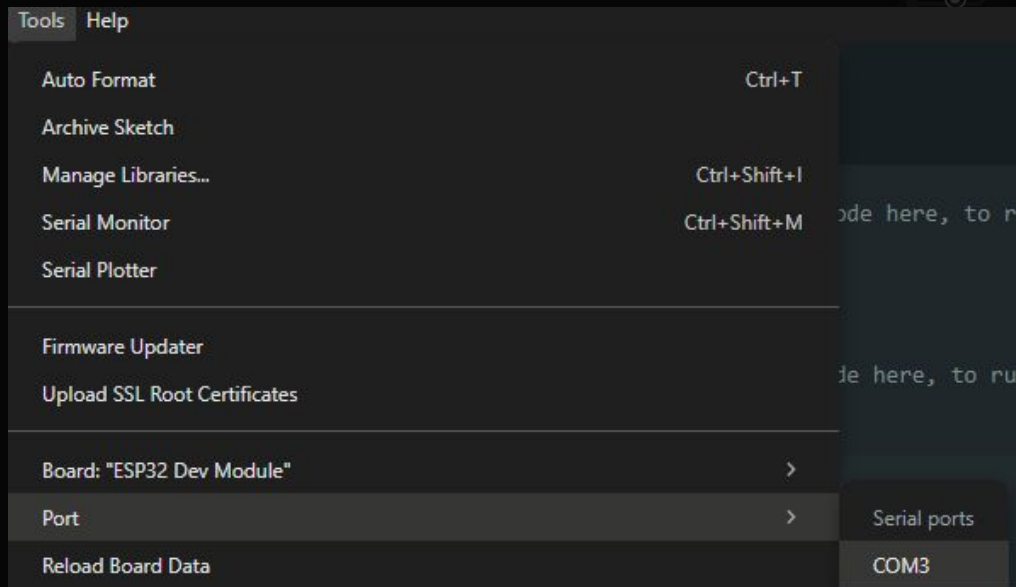
The Display in Session 2 will not work without it. :(

How to Install and Configure the Arduino IDE continued...



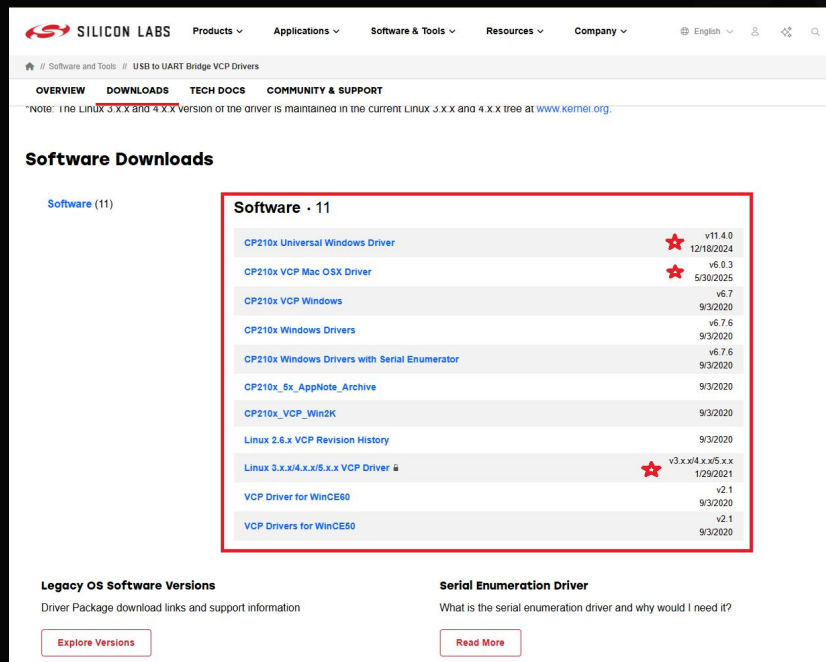
Go back to tools, then go the the board section, then hover over ESP32, then select “ESP32 Dev Module”

How to Install and Configure the Arduino IDE continued...



Then, once you have selected the correct board, you will see you finally have the option to select a Port. Select the COM3 Port.

How to Install and Configure the Arduino IDE continued...



SILICON LABS Products Applications Software & Tools Resources Company

Software and Tools # USB to UART Bridge VCP Drivers

OVERVIEW DOWNLOADS TECH DOCS COMMUNITY & SUPPORT

*Note: The Linux 3.x.x/4.x.x/5.x.x version of the driver is maintained in the current Linux 3.x.x and 4.x.x tree at www.kernel.org.

Software Downloads

Software (11)

Software · 11

CP210x Universal Windows Driver	★	v11.4.0 12/18/2024
CP210x VCP Mac OSX Driver	★	v6.0.3 5/30/2025
CP210x VCP Windows		v6.7 9/3/2020
CP210x Windows Drivers		v6.7.6 9/3/2020
CP210x Windows Drivers with Serial Enumerator		v6.7.6 9/3/2020
CP210x_5x_AppNote_Archive		9/3/2020
CP210x_VCP_Win2K		9/3/2020
Linux 2.6.x VCP Revision History		9/3/2020
Linux 3.x.x/4.x.x/5.x.x VCP Driver	★	v3.x.x/4.x.x/5.x.x 1/29/2021
VCP Driver for WinCE60		v2.1 9/3/2020
VCP Drivers for WinCE50		v2.1 9/3/2020

Legacy OS Software Versions

Driver Package download links and support information

[Explore Versions](#)

Serial Enumeration Driver

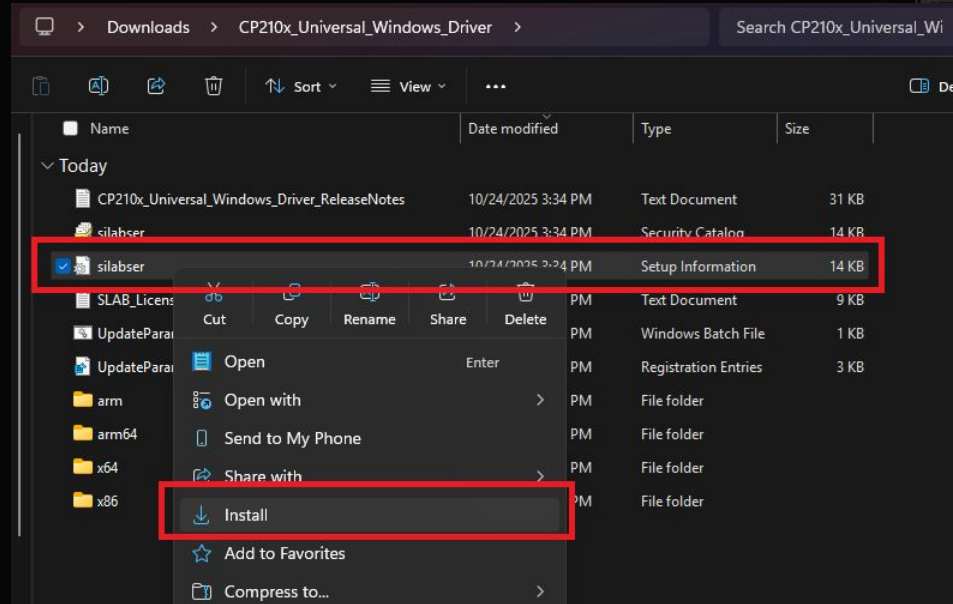
What is the serial enumeration driver and why would I need it?

[Read More](#)

Then download the appropriate driver based on your OS

Link: <https://www.silabs.com/software-and-tools/usb-to-uart-bridge-vcp-drivers?tab=downloads>

How to Install and Configure the Arduino IDE continued...

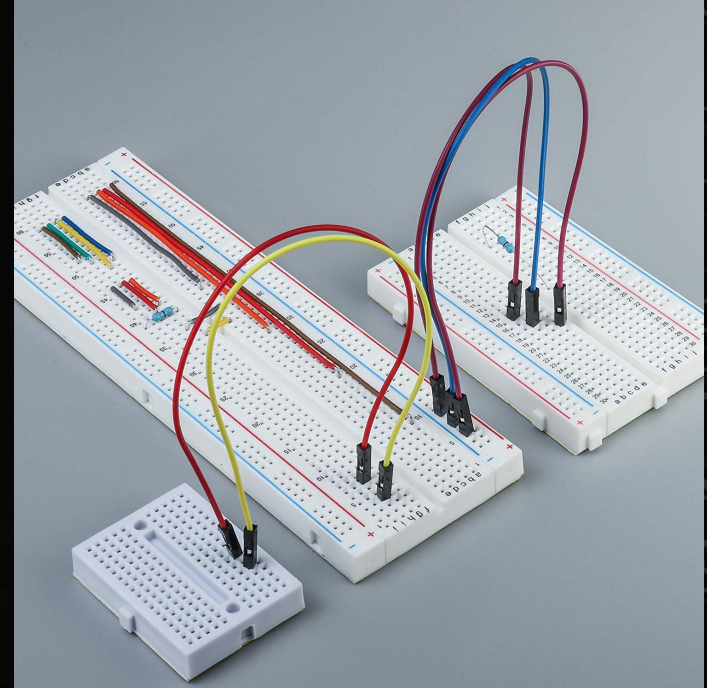


Extract the folder > Run the “silabser” .inf/setup information file installer (right-click and select “install”)

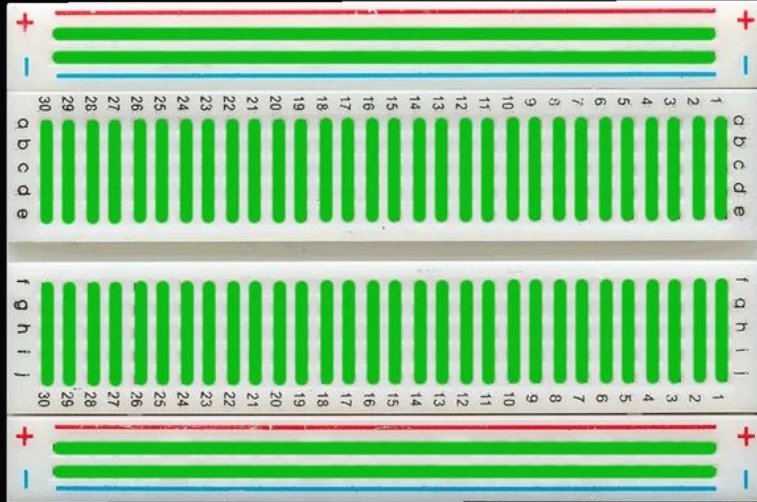
And then you're done! :)

How to Connect the Components on the Breadboard!

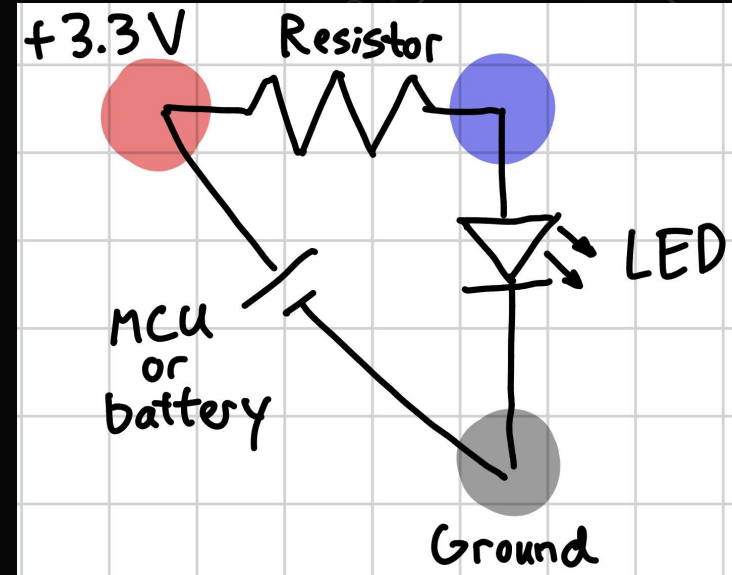
- So.. what is a solderless breadboard?
 - It's a tool that allows us to connect our MCU (microcontroller) and components easily.
- But how do we connect things?
 - Certain rows/columns of the breadboard are connected, we plug wires into the same rows/columns to connect them!



How to Use the Breadboard Continued..

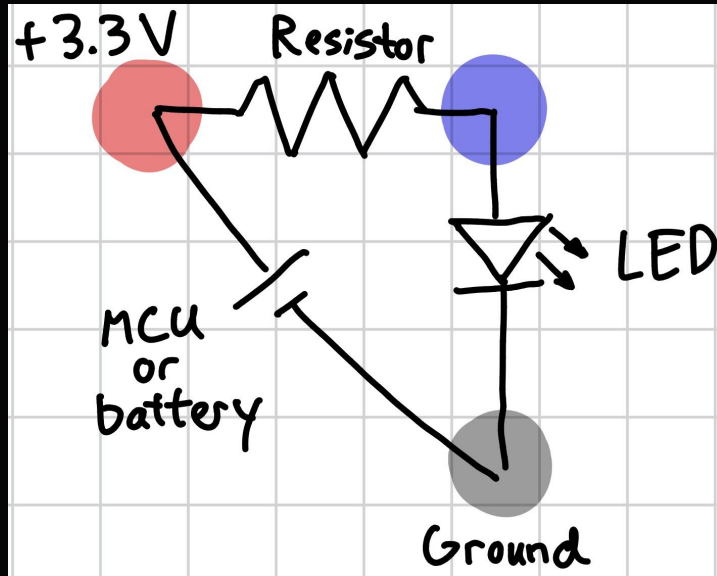


The green lines show the connections on the breadboard

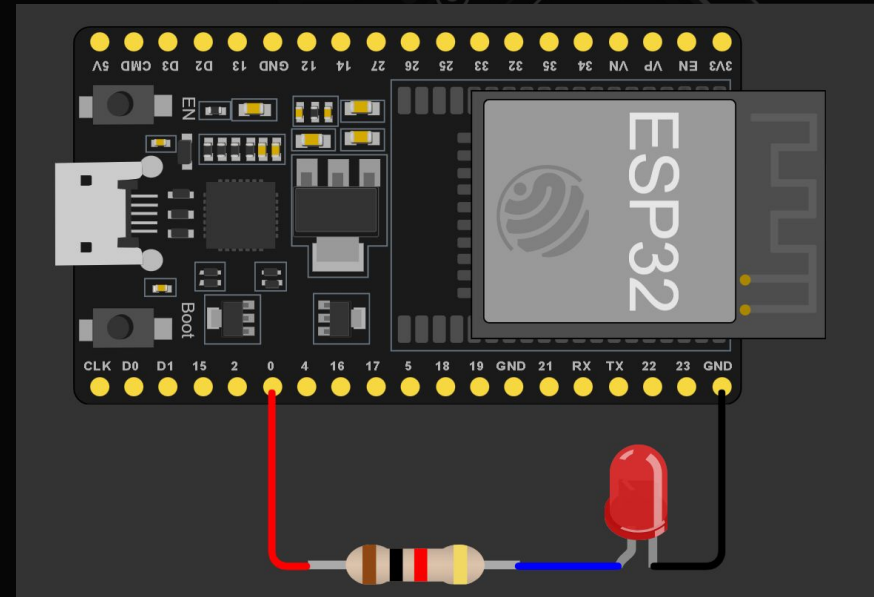


Circuit schematics show the components in their symbol form and their connections

How to Use the Breadboard Continued..

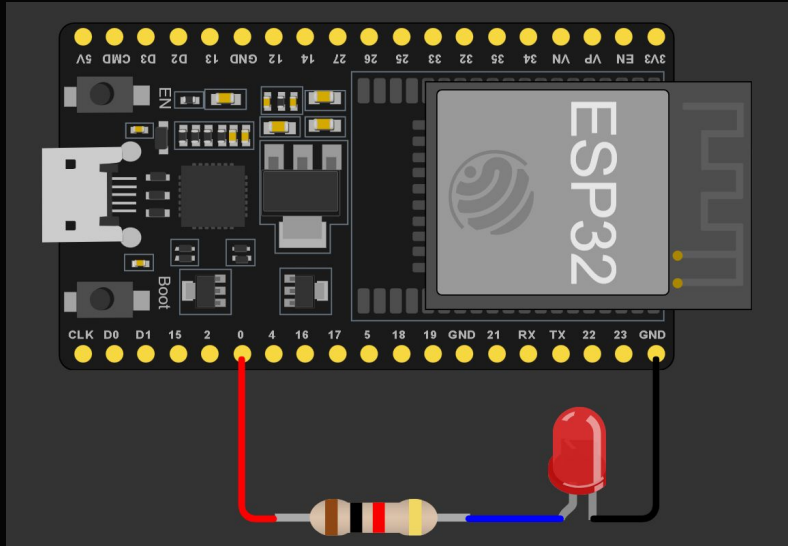


Circuit schematics show the components in their symbol form and their connections

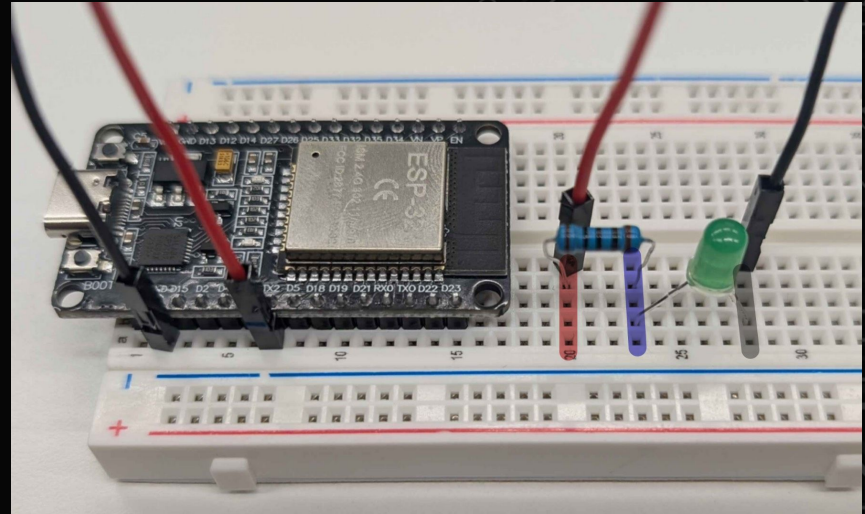


We can realize these connections using our actual components by soldering or custom PCBs

How to Use the Breadboard Continued..



We can realize these connections using our actual components by soldering or custom PCBs

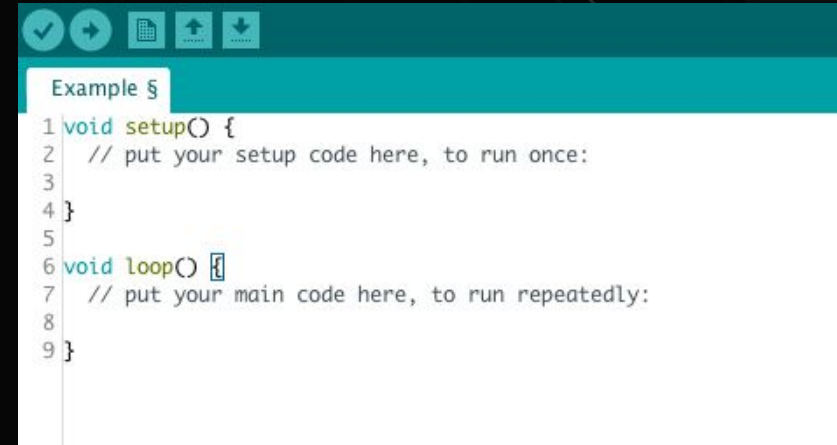


The breadboard can be used instead to make the connections easy!
*Note the highlighted lines

Structure of an Embedded Program

There are two main phases of an Embedded Program:

- **setup() - Initialization (runs once at startup!)**
 - Configure pin directions (input/output)
 - Initialize peripherals: UART, SPI, I²C, ADC, PWM, etc..
 - Start timers, interrupts, and libraries
 - Prepare sensors or communication modules
- **loop() – Execution (runs continuously)**
 - Read sensors and update variables
 - Run control logic (if/else, PID, FSMs, etc.)
 - Send or display data (serial, LCD, LEDs)
 - React to events and adjust outputs in real time

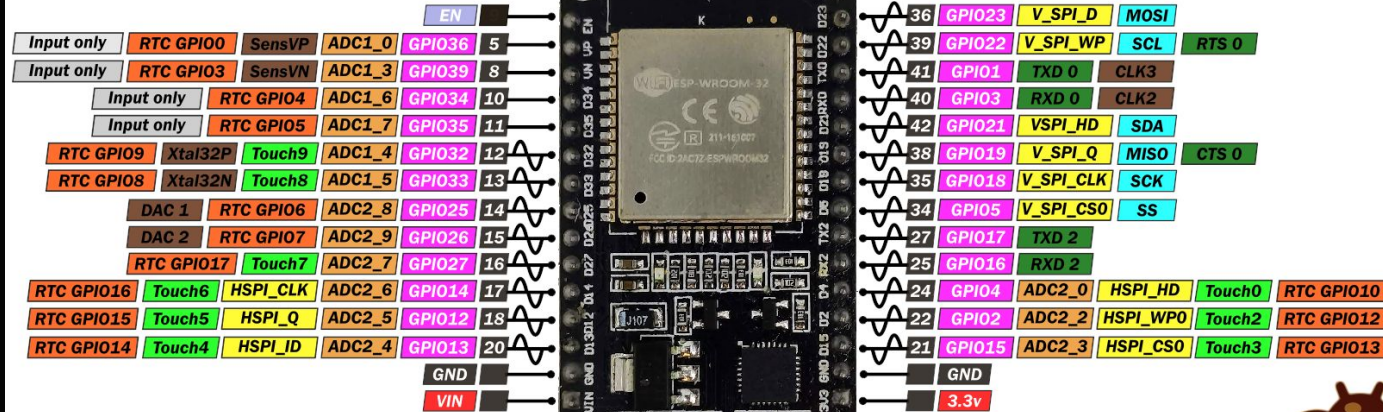


```

Example §
1 void setup() {
2   // put your setup code here, to run once:
3
4 }
5
6 void loop() {
7   // put your main code here, to run repeatedly:
8
9 }
  
```

ESP32 Pinouts

ESP32 DEV KIT V1 PINOUT



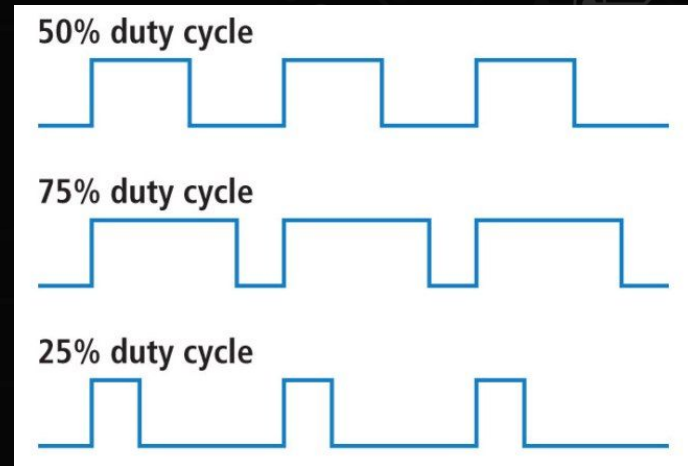
www.mischianti.org

(cc) BY-NC-ND



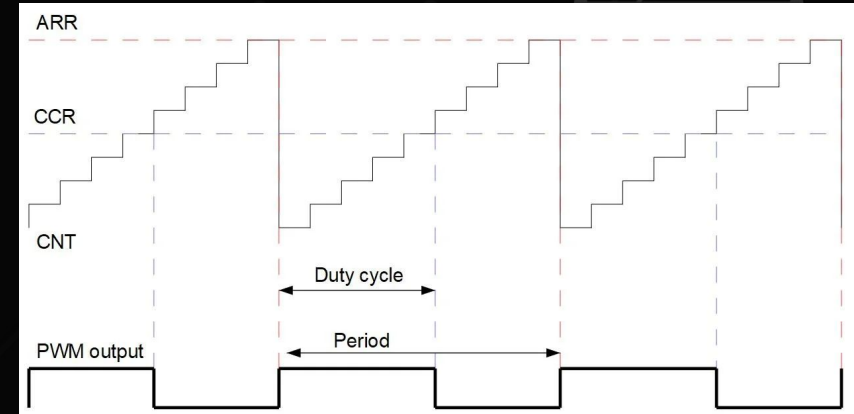
Pulse Width Modulation (PWM)

- How do we dim an LED? An easy method is Pulse Width Modulation (PWM)
- PWM is a type of modulation in which the width of a pulse that occurs at a constant interval is modified to become larger to thinner.
- The interval between pulses remains the same, but pulse width changes
- The **duty cycle** of a PWM signal refers to how long a pulse stays on (voltage high) in a single pulse cycle
- For an LED, the larger the duty cycle, the brighter the LED.



Timers

- How can we implement PWM? We can use timers
- Timers are hardware units within an MCU that use one of the various clocks on the ESP32 to count up every clock cycle to a specified limit
- The timer then raises a **flag**, an indicator that the limit was reached and therefore a corresponding amount of time has occurred
- This flag can be used to indicate to the ESP32 to perform certain actions, such as setting a pin high or low at regular intervals to create a PWM signal.



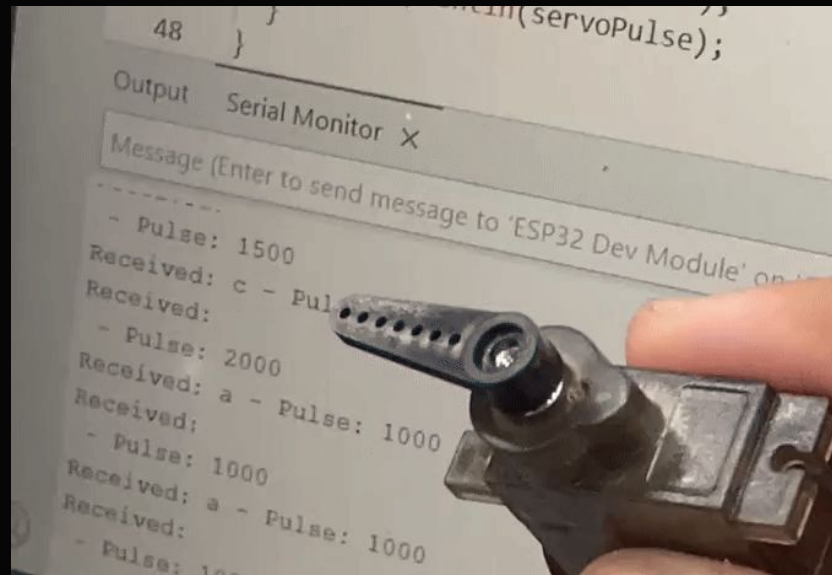
Polling and Interrupts

- How can we detect flags? There are two methods: polling and interrupts.
- **Polling** is essentially writing in a while loop to keep checking if a flag has changed and to proceed with the needed actions.
 - Polling is highly CPU inefficient, wasting time on constantly checking if an event occurred.
- **Interrupts** are requests to a CPU to literally interrupt its regular execution, save its state, and perform a necessary action.
- Special functions named Interrupt Service Routines (ISRs) outline these actions.
- In two routines (or polling loops), we can write the code to toggle the PWM.

```
void IRAM_ATTR handleButtonPress() {  
    // Toggle the LED state  
    // This code is for a button not a timer  
    // But it is similar!  
    digitalWrite(ledPin, !digitalRead(ledPin));  
}
```

Let's look at a real example using interrupts!

UART Interrupts with Servo Control



Example Code #1

```
#include <Arduino.h>
#include "driver/uart.h" // this gives us direct access to ESP32's UART hardware driver

#define SERVO_PIN 32 // GPIO32 for servo signal
#define LED_PIN 2    // onboard LED (used to show interrupt activity)

#define UART_NUM UART_NUM_0 // we're using UART0 (the same one used for Serial Monitor)
#define BUF_SIZE 128        // buffer size for incoming serial data

volatile int servoPulse = 1500; // starting servo position (1.5ms = center)
volatile bool servoUpdate = false; // flag that tells loop() when to move servo

QueueHandle_t uart_queue; // queue to hold UART interrupt events (FreeRTOS thing)
```

Example Code #2

```
// this runs in the background, triggered by UART interrupts
void uartEventTask(void *pvParameters) {
    uart_event_t event; // stores info about what kind of UART event happened
    uint8_t data[BUF_SIZE]; // buffer for received bytes

    while (1) {
        // wait here until a UART event happens (interrupt tells us)
        if (xQueueReceive(uart_queue, (void *)&event, portMAX_DELAY)) {
            if (event.type == UART_DATA) { // if it's incoming data...

                // read however many bytes came in
                int len = uart_read_bytes(UART_NUM, data, event.size, portMAX_DELAY);

                // loop through each character we got
                for (int i = 0; i < len; i++) {
                    char c = (char)data[i]; // convert byte to character
                }
            }
        }
    }
}
```

Example Code #3

```
// move servo based on input
if (c == 'a') {
    servoPulse = 1000; // 0 degrees
    servoUpdate = true;
}
else if (c == 'b') {
    servoPulse = 1500; // 90 degrees
    servoUpdate = true;
}
else if (c == 'c') {
    servoPulse = 2000; // 180 degrees
    servoUpdate = true;
}

// LED toggles every time we receive a character
digitalWrite(LED_PIN, !digitalRead(LED_PIN));
}
}
}

// helper function to actually update servo PWM signal
void updateServo(int pulseWidth) {
    // so servo expects a 20ms signal period (50Hz)
    // here we convert pulse width (like 1500us) into PWM duty cycle
    uint32_t duty = (pulseWidth * 65536) / 20000; // (pulse / period) * resolution
    ledcWrite(SERVO_PIN, duty); // update the PWM duty cycle on that pin
}
```

Example Code #4

```
void setup() {
    pinMode(LED_PIN, OUTPUT); // LED shows when interrupts happen

    // setup the PWM output for the servo
    ledcAttach(SERVO_PIN, 50, 16); // 50Hz, 16-bit resolution
    updateServo(servoPulse); // center servo at startup

    // now we configure the UART hardware
    uart_config_t uart_config = {
        .baud_rate = 115200,
        .data_bits = UART_DATA_8_BITS,
        .parity = UART_PARITY_DISABLE,
        .stop_bits = UART_STOP_BITS_1,
        .flow_ctrl = UART_HW_FLOWCTRL_DISABLE,
        .source_clk = UART_SCLK_DEFAULT,
    };

    // this installs the UART driver and sets up the interrupt queue
    uart_driver_install(UART_NUM, BUF_SIZE * 2, BUF_SIZE * 2, 20, &uart_queue, 0);
    uart_param_config(UART_NUM, &uart_config);

    // creating a background FreeRTOS task to handle UART interrupts
    xTaskCreate(uartEventTask, "uart_event_task", 2048, NULL, 12, NULL);

    Serial.println("UART INTERRUPT DEMO");
    Serial.println("This uses UART interrupts to handle Serial input instantly.");
    Serial.println("Send 'a', 'b', or 'c' to move the servo.");
    Serial.println("LED blinks every time a character is received.");
}
```

Example Code #5

```
void loop() {
  // loop just checks if the interrupt task set our flag
  if (servoUpdate) {
    servoUpdate = false; // reset flag
    updateServo(servoPulse); // actually move servo

    // print confirmation so students can see it react in Serial Monitor
    Serial.print("UART Interrupt!! Pulse: ");
    Serial.print(servoPulse);
    Serial.println(" microseconds");
  }

  // note: loop() isn't constantly checking Serial, interrupts handle that!
}
```

QR Codes!!!!



[Feedback Link](#)



[IEEE Discord Link](#)